

Instant Graph Neural Networks for Dynamic Graphs

Yanping Zheng
Renmin University of China
Beijing, China
zhengyanping@ruc.edu.cn

Hanzhi Wang
Renmin University of China
Beijing, China
hanzhi_wang@ruc.edu.cn

Zhewei Wei*
Renmin University of China
Beijing, China
zhewei@ruc.edu.cn

Jiajun Liu
Data 61, CSIRO
Queensland, Australia
jiajun.liu@csiro.au

Sibo Wang
The Chinese University of Hong Kong
Hong Kong, China
swang@se.cuhk.edu.hk

ABSTRACT

Graph Neural Networks (GNNs) have been widely used for modeling graph-structured data. Recent breakthroughs have been made in improving the scalability of GNNs to work on graphs with millions of nodes. However, how to instantly represent continuous changes of large-scale dynamic graphs with GNNs is still an open problem. Existing dynamic GNNs focus on modeling the periodic evolution of graphs, often on a snapshot basis. Such methods suffer from two drawbacks: first, there is a substantial delay for the changes in the graph to be reflected in the graph representations, resulting in losses on the model's accuracy; second, repeatedly calculating the representation matrix on the entire graph in each snapshot is predominantly time-consuming and severely limits the scalability. In this paper, we propose Instant Graph Neural Network (*InstantGNN*), an incremental computation approach for the graph representation matrix of dynamic graphs. Set to work with dynamic graphs with the edge-arrival model, our method avoids time-consuming, repetitive computations and allows instant updates on the representation and instant predictions. Graphs with dynamic structures and dynamic attributes are both supported. The upper bounds of time complexity of those updates are also provided. Furthermore, our method provides an adaptive training strategy, which guides the model to retrain at moments when it can make the greatest performance gains. We conduct extensive experiments on several real-world and synthetic datasets. Empirical results demonstrate that our model achieves state-of-the-art accuracy while having orders-of-magnitude higher efficiency than existing methods.

CCS CONCEPTS

• Theory of computation → Dynamic graph algorithms; • Information systems → Data mining.

*Zhewei Wei is the corresponding author. The work was partially done at Gaoling School of Artificial Intelligence, Peng Cheng Laboratory, Beijing Key Laboratory of Big Data Management and Analysis Methods and MOE Key Lab of Data Engineering and Knowledge Engineering.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).
KDD '22, August 14–18, 2022, Washington, DC, USA

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9385-0/22/08...\$15.00
<https://doi.org/10.1145/3534678.3539352>

KEYWORDS

graph neural networks, dynamic graphs

ACM Reference Format:

Yanping Zheng, Hanzhi Wang, Zhewei Wei, Jiajun Liu, and Sibow Wang. 2022. Instant Graph Neural Networks for Dynamic Graphs. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22)*, August 14–18, 2022, Washington, DC, USA. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3534678.3539352>

1 INTRODUCTION

Graph Neural Networks (GNNs) are at the frontier of graph learning research. They have been widely employed in various applications, such as recommendation systems [9], molecules synthesis and manipulation [37], etc. Traditional GNN algorithms, with their un-optimized messaging techniques and storage mechanics, struggle to handle large-scale graph data. Numerous methods [4, 7, 13, 33, 34, 38] have been proposed to improve the scalability of GNNs and optimize the efficiency in processing large-scale graphs, by exploiting the computational characteristics of GNNs. These algorithms are designed for static graphs and use static computation strategies. However, graphs in the real-world generally change their graph structure and properties over time. Several dynamic Graph Neural Networks [10, 24, 25, 41, 42] have been proposed in recent years to model the temporal information from graph evolution. Nevertheless, they focus on the prediction of the properties of nodes in the future and are limited in scalability. How to instantaneously compute the representation matrix on large-scale dynamic graphs remains an open problem.

Motivation. A GNN's working mechanism can be characterized by two operations: propagation and prediction. The former digests the message passing while the latter learns the weighted matrix. After a series of GNNs [17, 31] with both operations combined and being inseparable, SGC [34] and PPNP [18] signaled the finding that desirable properties could be introduced by separating the propagation process from the prediction process. By convention, a well-designed message propagation mechanism can be substantially beneficial to GNN models [43].

Let Z be the graph representation matrix, a generalized propagation equation which has been used in SGC [34], AGP [33] and GBP [7] can be formulated as:

$$Z = \sum_{\ell=0}^{\infty} \alpha(1-\alpha)^{\ell} \mathbf{P}^{\ell} \mathbf{X} = \sum_{\ell=0}^{\infty} \alpha(1-\alpha)^{\ell} \left(\mathbf{D}^{-a} \mathbf{A} \mathbf{D}^{-b} \right)^{\ell} \mathbf{X}, \quad (1)$$

where $\mathbf{X}$ denotes the feature matrix of the input graph, $\mathbf{A}$ and $\mathbf{D}$ represent the adjacent matrix and the degree matrix, respectively. The

propagation matrix $\mathbf{P}$, defined as $\mathbf{D}^{-a}\mathbf{A}\mathbf{D}^{-b}$, can easily generalize existing methods, where a and b are Laplacian parameters ranging from 0 to 1. By setting $a = b = 1/2$, $\mathbf{P}$ represents the symmetric normalization adjacency matrix used in [17, 18, 34]. Also, the transition probability matrix $\mathbf{A}\mathbf{D}^{-1}$ [8, 38] and the reverse transition probability matrix $\mathbf{D}^{-1}\mathbf{A}$ [35] are obtained by setting $a = 0, b = 1$ and $a = 1, b = 0$, respectively. After deriving $\mathbf{Z}$, we can utilize Multi-Layer Perceptrons (MLPs) to make the prediction with mini-batch training. Therefore, a GNN model for the multi-class classification task can be written as $\mathbf{Y} = \text{softmax}(\text{MLP}(\mathbf{Z}))$.

It is known that the calculation of $\mathbf{Z}$ is computationally expensive. To handle dynamic graphs, static or snapshot-based GNN models have to recompute to obtain the updated representation matrix $\mathbf{Z}$ periodically, which increases computational complexity dramatically. A natural question worth investigating is “*can we reuse results from previous computations to accelerate the calculation process for the current moment?*”. This paper aims to propose a dynamic GNN algorithm that incrementally updates the representation matrix in Equation 1 as the graph changes over time, with necessary updates on affected nodes only, avoiding the time-consuming computation on the entire graph.

Contributions. In this paper, we present InstantGNN, an efficient approximate GNN on dynamic graphs. The intuition behind our method is to incrementally update the representation matrix based on the generalized propagation framework. Specifically, we adopt the computing framework of Personalized PageRank to obtain an approximate version of the representation matrix $\mathbf{Z}$, which is inspired by [4, 33]. When the graph changes, we adjust the representation vectors of the affected nodes locally, and the impact of the change on remaining nodes is calculated by propagation operation in the next step. As a result, our method provides instantaneous updates for node representations while guaranteeing the same error as the recalculation. The basis of the update strategy is based on the underlying mechanics of Personalized PageRank, i.e., we always maintain that the invariant holds throughout the evolution of dynamic graphs, as described in Lemma 1. Extensive experimental results on real-world and synthetic datasets demonstrate the effectiveness and efficiency of InstantGNN. Especially, InstantGNN improves the computing efficiency of large graphs with millions of nodes by one-to-several orders of magnitude, while keeping comparable accuracy. Our contributions can be summarised as follows:

- We propose an Instant Graph Neural Network model for dynamic graphs, namely *InstantGNN*. Our method is able to compute the graph representation matrix incrementally based on the result of previous steps, avoiding unnecessary re-computation and with guaranteed accuracy. Instead of just the dynamic graph structure, the method deals with dynamic node features too.
- We provide strong guarantees on the total running time of computing and updating the approximate propagation matrix on undirected graphs, as shown in Theorem 3&4.
- Most previous work is exclusively evaluated on graphs with static node labels. We construct a real-world dataset from Aminer Network coauthor data, which contains both dynamic structure and dynamic node labels.
- Our method can create an adaptive forecast for the optimal time to retrain the model, which is useful for maximizing performance gains with a fixed budget for computational costs.

Table 1: Table of notations

Notation	Description
$G = (V, E)$	the graph with vertex set V and edge set E
$\mathbf{x}$	the graph signal vector
$\boldsymbol{\pi}, \hat{\boldsymbol{\pi}}$	the true and estimated propagation vectors
$\mathbf{r}$	the residual vector
$\mathbf{e}_s$	the one-hot vector with $\mathbf{e}_s(s) = 1$
$d(s)$	the degree of node s
$N(s)$	the neighbor set of node s
M	the graph event set, $M = \{Event_1, \dots, Event_k\}$
G_i, E_i	the graph and edge set at time i ($0 \leq i \leq k$)
$\mathbf{A}_i, \mathbf{D}_i$	the adjacency matrix and degree matrix of G_i

2 PRELIMINARIES

We consider an undirected graph $G = (V, E)$, where V is the vertex set with $n = |V|$ nodes, E denotes the edge set with $m = |E|$ edges. Detailed notations are shown in Table 1.

2.1 Dynamic Graphs

In general, dynamic graphs can be broadly classified into two categories: Continuous-Time Dynamic Graph (CTDG) and Discrete-Time Dynamic Graph (DTDG). Given an initial graph G_0 , a CTDG describes a continuous graph evolving process which can be formulated as sequential events $\{Event_1, \dots, Event_k\}$, where k is the number of graph events. Specifically, $Event_i = (type_i, e_i)$ denotes an edge operation of $type_i$ conducting on edge $e_i \in E_i$ at time i . The types of edge operations are restricted as *InsertEdge* and *DeleteEdge*. For the sake of simplicity, we assume the graph evolves from time $i = 0$ and only one edge operation is conducted on one edge at each time step. More precisely, let G_i denote the graph topology at the time step i . Then G_i can be derived after i subsequent edge operations (i.e. $\{Event_1, \dots, Event_i\}$) from G_0 . A DTDG is a sequence of static graph snapshots taken at the time intervals, represented as $S = \{G_0, \dots, G_{|S|}\}$. A graph snapshot G_i in a DTDG can be treated as the graph captured at time i of a CTDG, while the detailed evolving process (e.g. $\{Event_1, \dots, Event_i\}$) is omitted. Therefore, CTDGs are more general than DTDGs [16], and we aim to design GNN algorithms for CTDGs.

Dynamic Attributes. We note that for real-world graphs, both graph topologies and node attributes change over time. For example, consider the paper collaboration networks with authors as graph nodes and the research interests each author have as the node attribute. The changes of authors’ research interests result in dynamic node attributes. Thus, in Section 4.3 and 4.4, we consider the CTDG with dynamic node attributes (DA-CTDG). Specifically, we use $\mathbf{X}_i$ to denote the node feature matrix at the time step i . And $\mathbf{X}_i(s)$ corresponds to the attribute vector of node s .

Problem Definition. We consider a dynamic attribute graph with a fixed node set V , a dynamic edge set, and a dynamic feature matrix. Let $\mathbf{Z}_i$ denote the graph representation matrix of the graph G_i , $\mathbf{Z}_{i+1}$ is the matrix after operating $Event_{i+1}$ or updating node features to $\mathbf{X}_{i+1}$. The problem of InstantGNN is to calculate the representation matrix $\mathbf{Z}_{i+1}$ incrementally and instantly.

2.2 Personalized PageRank (PPR) based GNNs

The mainstream methods of propagation in GNNs are implemented based on Personalized PageRank (PPR). PageRank (PR) [23] is a

web ranking algorithm used by Google, and PPR is its personalized version. PPR measures the importance of nodes from the point of view of a given node, and it has been widely used in web search engines [15, 23] and social recommendations [12]. Given a graph G and a starting node s , the PPR score reflects the relative importance of node $t \in V$ with respect to node s . Formally, the PPR is calculated as follows:

$$\pi_s = \alpha e_s + (1 - \alpha) \mathbf{A} \mathbf{D}^{-1} \pi_s, \quad (2)$$

where α is the teleport probability and $\alpha \in (0, 1)$, e_s is a one-hot vector with $e_s(s) = 1$. By solving Equation 2 and using a power series expansion, an equivalent form is given by $\sum_{\ell=0}^{\infty} \alpha(1-\alpha)^\ell (\mathbf{A} \mathbf{D}^{-1})^\ell e_s$.

Efficient methods for calculating PPR have been extensively studied. A commonly used approximate version is known as Forward Push [3], which focuses on the region close to the source nodes and avoids traversing the entire graph. Based on the significant performance of PPR in applications such as recommendations, researchers found that PPR is incredibly beneficial for understanding and improving graph neural networks [18] recently. Many methods, including PPRGo [4], GBP [7], and AGP [33], provide enhanced propagation strategies based on approximation PPR, in order to accelerate the propagation process and achieve efficient computing of GNNs. Similarly, the method proposed in this paper adopts and extends the approximate dynamic PPR [40] to solve the GNN propagation problem in dynamic graphs.

3 RELATED WORK

3.1 Scalable GNNs

Due to the high complexity of traditional full-batch GNN approaches, they have difficulty supporting large-scale graphs with millions of nodes. Numerous methods have been introduced to tackle this problem, which can be summarized into two categories: sampling-based methods and linear models.

Sampling-based methods divide a large graph into smaller segments by sampling subsets of neighbors or sub-graphs of the original graph. GraphSage [13] is the first to consider the scalability of graph neural networks, in which each node only aggregates the information from its sampled neighbors. Another node sampling approach, VR-GCN [5], further reduces the sampling size but still does not solve the problem of exponential growth in the number of neighboring nodes. Layer sampling approaches, such as FastGCN [6] and ASGCN [39], successfully expand to large-scale graphs by sampling a fixed number of nodes at each layer of the GCN. However, overlapping nodes in different batches introduce significant computational redundancy, reducing the method's efficiency. Graph sampling methods represented by ClusterGCN [8] and GraphSAINT [38] obtain a series of subgraphs of the original graph by exploiting the graph clustering structure or subgraphs sampler. The problem of exponential growth of neighbors can be remedied by restricting the size of subgraphs.

Linear models eliminate the nonlinearity between layers in the forward propagation, allowing us to combine different neighboring aggregators in a single layer and design scalable models without relying on samplings. SGC [34] and PPNP [18] discover that the propagation and neural network (activations) can be separated in GNNs and therefore open the door for simpler or approximate methods. GDC [19] and GBP [7] capture information about multi-hop

neighborhoods using PPR and heat kernel PageRank and convert the initial feature matrix to a representation matrix for subsequent tasks. PPRGo [4] calculates the approximate PPR matrix by pre-computation and then applies the PPR matrix to the feature matrix, deriving the final propagation matrix. The precomputation deals with all message passing operations on the graph topology.

3.2 Dynamic Graph Representation Learning

The straightforward way to handle dynamic graphs is to apply the traditional static GNN models, for example, GCN [17], GAT [31] and GraphSage [13], to learn the node representation on every graph snapshot. However, generating new representation at each time step is computationally costly, since most traditional graph neural network methods learn node representation by retraining the whole model. Some researchers proposed to combine temporal modeling with node representation learning. DySAT [26] learns node representations on each graph snapshot and employs attention mechanisms to learn structural and temporal attention to integrate useful information into node representations adaptively. To this end, a common solution is to use graph learning algorithms to collect structural information and utilize sequence models, such as Recurrent Neural Networks (RNNs), to capture temporal dependencies and add consistency constraints between adjacent time steps [10, 11, 21, 25, 28]. These methods view a dynamic graph as a discrete evolution process. The graph is sampled with an interval into snapshots which consist of a large number of graph changes instead of a continuous stream of atomic graph events that consistently change the graph.

Nguyen et al. [22] suggest modeling the spatio-temporal network as a continuous model. They add a temporal constraint to random walk so that both the structural and temporal patterns of the nodes can be captured. Similarly, HTNE [44] introduces the neighborhood formation to describe the evolutionary process of nodes and applies the Hawkes process to capture the influence of historical neighbors on nodes. Through an attention mechanism, DyRep [30] creates an RNN-like structure, embedding structural information about temporal evolution into node representations. And JODIE [20] proposes a coupled recurrent neural network architecture and applies two independent RNNs to update the embeddings of users and items.

Limitation. To the best of our knowledge, the problem defined in Section 2.1 has not been addressed in the existing literature. Existing scalable graph neural networks are unfortunately inefficient for our problem as they are optimized for static graphs [4, 33]. They assume static graphs and compute representation matrixes from scratch whenever a graph is updated. Since the graph is updated frequently in dynamic environments, it is computationally expensive or even unviable to employ such techniques. On the other hand, temporal GNNs and network embedding focus on modeling evolution rather than enhancing the efficiency of generating node representations, and they are challenging to scale to large graphs.

4 INSTANT GRAPH NEURAL NETWORK

In real-world graph-like structures, the relations between nodes may change at any time, which means that graphs evolve continuously and, in certain cases, are never-ending. It is sub-optimal to model these changes with discretely sampled graph snapshots, as firstly changes will not be reflected in the representations in time,

and secondly the full model needs to be retrained from the entire graph at each time step, raising efficiency and scalability issues. To address this problem, we use incremental, approximate learning for GNN, which effectively handles the continuous graph changes and make predictions with instantly updated representations. Our approach is essentially a scalable and online graph neural network. Scalability is achieved by decoupling the propagation from the prediction progress, allowing the model to be trained in small batches and scaled to large graphs. Then we update graph representation vectors for necessary nodes only to attain real-time.

In this section, we present an instant propagation method, which can be recognized as an extension for dynamic graphs based on the static method. For ease of understanding, we first introduce the basic propagation method. In Section 4.2, we describe details of the instant propagation method, especially how to calculate increments for CTDGs. Later Section 4.3 introduces the extension of our algorithm to DA-CTDGs. A theoretical guarantee of the running time is provided in Section 4.4. Finally, we propose an adaptive training strategy based on InstantGNN in Section 4.5.

4.1 Basic Propagation Method

Although our method is naturally applicable to general circumstances, for the sake of simplicity, we restrict $a + b = 1$ in Equation 1 and set $\beta = a$, $1 - \beta = b$ in this paper. Treating $X \in \mathbb{R}^{(n \times d)}$ as d independent information vectors x , Z can be split into d segments, with each segment denoted by π . Therefore, we can write Equation 1 in a vector form: $\pi = \sum_{\ell=0}^{\infty} \alpha(1 - \alpha)^\ell P^\ell x$. Theoretically, π has the effect of aggregating the information of all nodes on the graph using infinite hops (GNN layers), but computation time and training parameters are reduced. However, the summation goes to infinity, making it computationally infeasible to calculate the result exactly on large graphs. Following [4, 33], we consider an approximate version of π as vector $\hat{\pi}$, which is the approximate solution under a given error limit ε . Meanwhile, we maintain a vector r with each element indicating the accumulated mass of information the node has received but not yet propagated. This enables us to design an approximate propagation process that achieves a speed-accuracy trade-off by adjusting r . Algorithm 1 describes the pseudo-code of the basic propagation progress. The algorithm takes the initial $\hat{\pi}$ and r as input. Then, we iteratively select the node s that exceeds the allowed error. Its α -fraction of the residual mass is converted into the estimated mass, and the remaining $((1 - \alpha) \cdot r(s))$ is distributed evenly among its neighbors using the transition matrix P . For instance, let $\hat{\pi} = 0$ and $r = x$ be the initial parameters, where 0 is a zero vector, Algorithm 1 outputs the propagation result corresponding to the current graph structure. As a result, we obtain $\hat{\pi}$ as an estimator for the graph propagation vector π which satisfies the condition $|\hat{\pi}(s) - \pi(s)| \leq \varepsilon \cdot d(s)^{1-\beta}$ for each node $s \in V$. We add self-loops for all nodes, following the setting in GCN [17], which guarantees that the degree of node $s \in V$ is at least 1. Therefore, if node s is a dangling node, it only propagates and aggregates information of its own, such that $|\hat{\pi}(s) - \pi(s)| \leq \varepsilon \cdot d(s)^{1-\beta}$ holds.

Therefore, the relationship between x , π , $\hat{\pi}$ and r can be described as Lemma 1, which is the basic premise for calculating the increment when the graph changes, and we shall discuss it later. Due to the space limit, we only present the conclusion and defer detailed proof to the technical report [1].

Algorithm 1: Basic Propagation Algorithm.

Input : Graph G , error threshold ε , teleport probability α , convolutional coefficient β , initial estimated propagation vector $\hat{\pi}$, initial residual vector r
Output: Updated estimated propagation vector $\hat{\pi}$ and residual vector r

```

1 while exist  $s \in V$  with  $|r(s)| > \varepsilon \cdot d(s)^{1-\beta}$  do
2    $\hat{\pi}(s) \leftarrow \hat{\pi}(s) + \alpha \cdot r(s)$ ;
3   for each  $t \in N(s)$  do
4      $r(t) \leftarrow r(t) + \frac{(1-\alpha) \cdot r(s)}{d(s)^\beta d(t)^{1-\beta}}$ ;
5    $r(s) \leftarrow 0$ ;
6 return  $\hat{\pi}, r$ ;
```

LEMMA 1. For every graph signal vector x , let π be the true propagation vector, the estimated vector $\hat{\pi}$ and residual vector r satisfy the following invariant property during the propagation process:

$$\hat{\pi} + \alpha r = \alpha x + (1 - \alpha) P \hat{\pi}. \quad (3)$$

4.2 Instant Propagation Method

Given a continuous-time dynamic graph (G, M) with the static graph signal x , we first apply Algorithm 1 to obtain $\hat{\pi}_0$ and r_0 for the graph at time $i = 0$. The approximate dynamic propagation algorithm aims to maintain a pair of $\hat{\pi}$ and r while keeping errors within a limited range, i.e. $r(s) \leq \varepsilon \cdot d(s)^{1-\beta}$ for any node $s \in V$. The key to solving the challenge is to maintain Equation 3 in the dynamic graph setting. By locally adjusting the affected nodes (by each graph event) to keep the equation holding, we can obtain feasible estimates and residuals corresponding to the updated graph. We start from the invariant equation of Lemma 1 and rewrite it as an equivalent formulation that for each node $s \in V$, we have:

$$\hat{\pi}(s) + \alpha r(s) = \alpha x(s) + \sum_{t \in N(s)} \gamma(s, t) \quad (4)$$

where $\gamma(s, t) = \frac{(1-\alpha) \hat{\pi}(t)}{d(s)^\beta d(t)^{1-\beta}}$ refers to the element related to node t in the equation of node s .

Without loss of generality, let us suppose that we add a new edge to graph G in the next time step. Considering the new edge as (u, v) , we found that only nodes $u, v, w \in N(u)$, and $y \in N(v)$ do not satisfy Equation 4, since no other variables change except the degrees of node u and v . By substituting the s in Equation 4 with u , it is easy to obtain $\hat{\pi}(u) + \alpha r(u) = \alpha x(u) + \sum_{t \in N(u)} \gamma(u, t)$, which is failed after inserting edge (u, v) . This is caused by two changes on the right-hand side of the equation:

- In denominator of the summation term, $d(u)$ changes to $d(u) + 1$. An intuitive explanation is that the mass u pushed out would always be divided into several equal proportions for all neighbors of u . Since v is added to be a new neighbor, the proportion should change to $\frac{1}{(d(u)+1)^\beta}$ accordingly.
- A new term, which refers to $\gamma(u', v') = \frac{(1-\alpha) \hat{\pi}(v)}{(d(u)+1)^\beta (d(v)+1)^{1-\beta}}$, is added, where u' and v' are the updated nodes u and v with degree $d(u) + 1$ and $d(v) + 1$, respectively.

The updated equation of node u implies: $\hat{\pi}(u) + \alpha r'(u) = \alpha x(u) + \sum_{t \in N(u)} \gamma(u', t) + \gamma(u', v')$, where $r'(u)$ is the adjusted residual,

also known as $\mathbf{r}(u) + \Delta\mathbf{r}(u)$. Such that we can obtain an accurate increment for residual of node u by subtracting $\mathbf{r}(u)$ from $\mathbf{r}'(u)$. Furthermore, we can use $\hat{\pi}(u) + \alpha\mathbf{r}(u) - \alpha\mathbf{x}(u)$ to equivalently substitute $\sum_{t \in N(u)} \gamma(u, t)$. Then we finally obtain the result:

$$\Delta\mathbf{r}(u) = (\hat{\pi}(u) + \alpha\mathbf{r}(u) - \alpha\mathbf{x}(u)) \cdot \frac{d(u)^\beta - (d(u)+1)^\beta}{\alpha \cdot (d(u)+1)^\beta} + \frac{\gamma(u', v')}{\alpha}. \quad (5)$$

Note that we only update u 's residual here, and its estimate $\hat{\pi}(u)$ will be adjusted in the next step.

For each node $w \in N(u)$, we observe that the equation $\hat{\pi}(w) + \alpha\mathbf{r}(w) = \alpha\mathbf{x}(w) + \sum_{t \in N(w)} \gamma(w, t)$ turns to be invalid. It is also caused by the fact that the degree of node u has changed. Specifically, the element containing node u has changed to $\gamma(w, u')$, where $\gamma(w, u') = \frac{(1-\alpha)\hat{\pi}(u)}{d(w)^\beta \cdot (d(u)+1)^{1-\beta}}$. And we obtain the precise increment of node w by altering its residual $\mathbf{r}(w)$ in a same fashion: $\Delta\mathbf{r}(w) = \frac{(1-\alpha)\hat{\pi}(u)}{\alpha \cdot d(w)^\beta} \cdot \left(\frac{1}{(d(u)+1)^{1-\beta}} - \frac{1}{d(u)^{1-\beta}} \right)$.

Updates to the affected nodes v and $y \in N(v)$ are almost the same as above. As a result, we have made adjustments for all affected nodes to keep Equation 4 holding to these nodes. However, the residuals of a few nodes may be outside of the error range after the correction, i.e., above $(\varepsilon \cdot d(s)^{1-\beta})$ or below $(-\varepsilon \cdot d(s)^{1-\beta})$. We then invoke Algorithm 1 to help guarantee that the error of results is within the error tolerance. Due to the fact that majority of nodes already have residuals that meet the requirements of the given error, the running time is significantly less than the initialization time.

According to the procedure described above, the situation of removing an edge can be solved in a similar process. The incremental update procedure differs from the insertion case only in the update of the degree, i.e. it should be $d(u) - 1$ instead of $d(u) + 1$. Let G_i be the graph updated from G_0 with $\{Event_1, \dots, Event_i\}$, $i = 1, \dots, k$ indicates the time that $Event_i$ arrives. Algorithm 2 describes the pseudo-code of generating the estimated propagation vector $\hat{\pi}$ and residual vector $\mathbf{r}$ for a CTDG in the case that graph events arrive one by one in order. Each step in the outer loop of Algorithm 2 proceeds as follows, where $e_i = (u, v)$ denotes the edge arrived at the current time step and $\hat{\pi}$ denotes the node representation of a dimension at this step. First, each affected node $(u, v, w \in N_{i-1}(u)$ and $y \in N_{i-1}(v))$ updates the residual to maintain its own equation, line 5-6 in Algorithm 2. Note that this update step may cause the error on some nodes to surpass the bounded range. Then, starting with the affected nodes, we invoke Algorithm 1 to reduce the error and propagate the effect of edge $e_i = (u, v)$ to the other nodes in the graph, line 7 in Algorithm 2.

Batch update. To extend Algorithm 2 to the batch update setting, given a set of graph events, we first update the graph and then calculate increments of residuals for the affected nodes. However, instead of iterating over all graph events, we compute only the final increments that are caused by the occurrence of k graph events on each affected node. Parallel processing of the affected nodes can further improve the efficiency of the algorithm. Appendix contains the complete pseudo-code.

4.3 Extension to Dynamic-attribute Graphs

This section discusses the extension of the proposed algorithm to handle DA-CTDGs, which are defined in Section 2.1. The challenge is that the structure and attribute of the graph are both changing over time. For ease of implementation, we first deal with the

Algorithm 2: Dynamic Propagation Algorithm.

Input : Dynamic graph $\{G_0, M = \{Event_1, \dots, Event_k\}\}$, graph signal vector $\mathbf{x}$, error threshold ε , teleport probability α , convolutional coefficient β

Output: estimated propagation vector $\hat{\pi}$, residual vector $\mathbf{r}$

```

1  $\hat{\pi} \leftarrow \mathbf{0}, \mathbf{r} \leftarrow \mathbf{x};$ 
2  $\hat{\pi}, \mathbf{r} \leftarrow \text{BasicPropagation}(G_0, \varepsilon, \alpha, \beta, \hat{\pi}, \mathbf{r});$ 
3 for  $Event_i = (e_i = (u, v), type_i) \in M$  and  $i = \{1, \dots, k\}$  do
4   Generate  $G_i$  from updating  $G_{i-1}$  by  $Event_i$ ;
5   UPDATE( $u, v, i, type_i$ );
6   UPDATE( $v, u, i, type_i$ );
7    $\hat{\pi}, \mathbf{r} \leftarrow \text{BasicPropagation}(G_i, \varepsilon, \alpha, \beta, \hat{\pi}, \mathbf{r});$ 
8 return  $\hat{\pi}, \mathbf{r};$ 
9 Function UPDATE( $node\ u, node\ v, time\ i, type$ ):
10   $\Delta\mathbf{r}(u) \leftarrow (\hat{\pi}(u) + \alpha\mathbf{r}(u) - \alpha\mathbf{x}(u)) \cdot \frac{d_{i-1}(u)^\beta - d_i(u)^\beta}{d_i(u)^\beta};$ 
11  if  $type$  is InsertEdge then
12     $\Delta\mathbf{r}(u) \leftarrow \Delta\mathbf{r}(u) + \frac{(1-\alpha)\hat{\pi}(v)}{d_i(u)^\beta d_i(v)^{1-\beta}};$ 
13  else if  $type$  is DeleteEdge then
14     $\Delta\mathbf{r}(u) \leftarrow \Delta\mathbf{r}(u) - \frac{(1-\alpha)\hat{\pi}(v)}{d_i(u)^\beta d_i(v)^{1-\beta}};$ 
15   $\Delta\mathbf{r}(u) \leftarrow \Delta\mathbf{r}(u)/\alpha;$ 
16   $\mathbf{r}(u) \leftarrow \mathbf{r}(u) + \Delta\mathbf{r}(u);$ 
17  for each  $w \in N_{i-1}(u)$  do
18     $\Delta\mathbf{r}(w) \leftarrow \frac{(1-\alpha)\hat{\pi}(u)}{\alpha \cdot d_i(w)^\beta} \cdot \left( \frac{1}{d_i(u)^{1-\beta}} - \frac{1}{d_{i-1}(u)^{1-\beta}} \right);$ 
19     $\mathbf{r}(w) \leftarrow \mathbf{r}(w) + \Delta\mathbf{r}(w);$ 
```

dynamic attributes, then employ Algorithm 2 to treat structural changes and obtain final results at each time step. The basic idea is to make local adjustments to ensure Equation 4 still holds for every node, which is similar to Section 4.2. The main difference is that every node could be an influenced node, since each element of the graph signal vector $\mathbf{x}$ may have changed.

We presume that the underlying graph structure is static while dealing with dynamic attributes. Let G_i be the graph at time i , where $0 \leq i \leq k$ and k is the number of graph events. We have $\mathbf{A}_0 = \mathbf{A}_1 = \dots = \mathbf{A}_k$, where $\mathbf{A}_i$ is the adjacent matrix of G_i . For each node $s \in V$, let $d_i(s)$ be the degree of node s at time i . Recall the definition, it is easy to observe that $d_0(s) = \dots = d_k(s)$. Let $\mathbf{x}_i$ be the graph signal vector of G_i , an intuition is to add the increment of the graph signal $\Delta\mathbf{x}_i = \mathbf{x}_i - \mathbf{x}_{i-1}$ to the residual vector $\mathbf{r}_{i-1}$, i.e. $\Delta\mathbf{r}_i = \Delta\mathbf{x}_i$. From Equation 4, for each node $s \in V$, the updated equation implies:

$$\hat{\pi}(s) + \alpha(\mathbf{r}(s) + \Delta\mathbf{r}(s)) = \alpha(\mathbf{x}(s) + \Delta\mathbf{x}(s)) + \sum_{t \in N(s)} \frac{(1-\alpha)\hat{\pi}(t)}{d(s)^\beta d(t)^{1-\beta}}.$$

As a result, setting $\Delta\mathbf{r}_i(s) = \Delta\mathbf{x}_i(s)$ is efficient to ensure that the equation holds for the graph G_i . After updating residuals for all impacted nodes, we can use the basic propagation algorithm to eliminate residuals exceeding the threshold until the desired error tolerance is reached. Following that, we can easily use Algorithm 2 by temporarily keeping the graph signal static.

4.4 Analysis

In this section, we provide a theoretical analysis of the computational complexity of the proposed method. All the proofs are deferred to the technical report [1] due to space constraints. We begin by providing an expected upper bound on the running time of CTDGs based on the method discussed in Section 4.2. Following that, we examine the time required to manage dynamic attributes. Therefore, the total running time for DA-CTDGs can be easily calculated by adding these two components. For illustration purposes, we define a vector $\Phi_i = \mathbf{1}^\top \cdot (\mathbf{D}_i^\beta \mathbf{Q}_i)$, where $\mathbf{1}$ is a vector with all elements set to 1, and $\mathbf{D}_i$ is the degree matrix of G_i . $\mathbf{Q}_i = \alpha[\mathbf{I} - (1 - \alpha)\mathbf{D}_i^{-\beta} \mathbf{A}_i \mathbf{D}_i^{\beta-1}]^{-1}$ is a matrix in which (s, t) -th element is equal to $d_i(s)^{1-\beta} \cdot \pi_{si}(t) \cdot d_i(t)^{\beta-1}$.

Cost for CTDGs. The total time cost is calculated by adding the initialization and the update time. We first present a theorem to bound the running time of step 2 of Algorithm 2.

THEOREM 1 (COST OF INITIALIZATION). *Given an error threshold ε , the time of generating an estimate vector $\hat{\pi}_0$ for the initial graph G_0 , such that $|\hat{\pi}_0(s) - \pi_0(s)| \leq \varepsilon \cdot d_0(s)^{1-\beta}$, for any $s \in V$, using Algorithm 1, is at most $T_{init} = \frac{1}{\alpha\varepsilon} \cdot (\|\mathbf{D}_0^\beta \cdot \pi_0\|_1 - \|\Phi_0^\top \cdot \mathbf{r}_0\|_1)$.*

Next, we will show that the time to maintain a pair of estimated propagation and residual vector meeting the error tolerance in the evolution is only related to the increment.

THEOREM 2 (COST OF UPDATES). *Let G_i denote the updated graph from G_{i-1} guided by Event_i . The time for updating $\hat{\pi}_{i-1}$ and $\mathbf{r}_{i-1}$, such that $|\hat{\pi}_i(s) - \pi_i(s)| \leq \varepsilon \cdot d_i(s)^{1-\beta}$, for any node $s \in V$, is at most $T_i = \frac{1}{\alpha\varepsilon} \cdot (2\varepsilon + \|\Phi_i^\top \cdot \Delta \mathbf{r}_i\|_1 + \|\Phi_{i-1}^\top \cdot \mathbf{r}_{i-1}\|_1 - \|\Phi_i^\top \cdot \mathbf{r}_i\|_1)$.*

Note that $\Delta \mathbf{r}_i$ is a vector with each element $\Delta \mathbf{r}_i(s)$, $s \in V$, corresponding to the precise increment of node s 's residual. We only adjust the residuals of the affected nodes, such that $\Delta \mathbf{r}_i$ is sparse with values only at $u, v, w \in N(u)$, $y \in N(v)$ when $e_i = (u, v)$. With the assumption that the edges arrive randomly and each node has an equal probability of being the endpoint, we have $E[\|\Phi_i^\top \cdot \Delta \mathbf{r}_i\|_1] = \frac{(6-4\alpha)\|\pi_{i-1}\|_1}{\alpha n} + \frac{6\varepsilon}{\alpha} - 2\varepsilon$, and details can be found in the technical report [1]. Now we consider the expected time required for running Algorithm 2.

THEOREM 3 (TOTAL COST). *Let $M = \{\text{Event}_1, \dots, \text{Event}_k\}$ be the graph event sequence. Given an error threshold ε , the time of generating an estimated vector $\hat{\pi}_k$ for the graph G_k , such that $|\hat{\pi}_k(s) - \pi_k(s)| \leq \varepsilon \cdot d_k(s)^{1-\beta}$, for any node $s \in V$, is at most $T = T_{init} + \sum_{i=1}^k T_i = \frac{\|\mathbf{D}_0^\beta \cdot \pi_0\|_1}{\alpha\varepsilon} + \frac{6k}{\alpha^2} + (6-4\alpha) \sum_{i=1}^k \frac{\|\pi_{i-1}\|_1}{\alpha^2 n \varepsilon}$.*

Thus, for the special case of $\beta = 0$, $\mathbf{x} = \mathbf{e}_s$, the worst-case expected running time is $O(\frac{1}{\varepsilon} + k + \frac{k}{n\varepsilon})$. Typically $\varepsilon = \Omega(\frac{1}{n})$, we can achieve $O(\frac{1}{\varepsilon} + k)$, where $O(\frac{1}{\varepsilon})$ is the expected time for initialization. Therefore, we can maintain estimates using $O(1)$ time per update.

Cost of handling dynamic attributes. Time complex analysis in this case is almost the same, the main difference is that the graph structure is static. Such that we have $\|(\Phi_i^\top - \Phi_{i-1}^\top) \cdot \mathbf{r}_{i-1}\|_1 = \varepsilon \cdot \sum_{s \in V} (d_i(s) - d_{i-1}(s)) = 0$. Note that $\Delta \mathbf{r}_i$ may not be a sparse vector here. Applying the increment mentioned above, we have $\|\Phi_i \Delta \mathbf{r}_i\|_1 = \sum_{s \in V} |\Phi_i(s) \Delta \mathbf{x}_i(s)| = \sum_{s \in V} |d_i(s)^\beta \Delta \mathbf{x}_i(s)|$. Therefore, we have Theorem 4 to bound the time cost of the proposed method at the dynamic setting that the graph signal is changing

Table 2: Statistics of experimental datasets.

Dataset	n	m	d	$ C $	$ S $
Arxiv	169,343	1,157,799	128	40	17
Products	2,449,029	61,859,012	100	47	16
Papers100M	111,059,956	1,615,685,872	128	172	21
Aminer	1,252,095	4,025,865	773	9	31
SBM-500K	500,000	6,821,393	256	50	10
SBM-1M	1,000,000	13,645,722	256	50	10
SBM-5M	5,000,000	68,228,223	1024	100	10
SBM-10M	10,000,000	136,469,493	1024	100	10

over time. It is clear that the expected time required to update estimates is proportional to the signal increment.

THEOREM 4 (TOTAL COST FOR HANDLING DYNAMIC ATTRIBUTES). *Let $\Delta \mathbf{x}_i$ be the increment of graph G_i updated from G_{i-1} . Given an error threshold ε , the time of generating an estimated vector $\hat{\pi}_k$ for the graph G_k , such that $|\hat{\pi}_k(s) - \pi_k(s)| \leq \varepsilon \cdot d_k(s)^{1-\beta}$, for any node $s \in V$, is at most $T = \frac{\|\mathbf{D}_0^\beta \cdot \pi_0\|_1}{\alpha\varepsilon} + \sum_{i=1}^k \frac{\|\mathbf{D}_i^\beta \Delta \mathbf{x}_i\|_1}{\alpha\varepsilon}$.*

Since the propagation process on each dimension is independent, we perform the dynamic propagation on multiple features in parallel, which further accelerates the execution of algorithm.

4.5 Adaptive Training

Typically, we retrain models at fixed intervals on continuous-time dynamic graphs, which is known as *Periodic training* and has been used in experiments of Sections 5.2 and 5.3. Let k and $|S|$ be the total number of graph events and times allowed retraining, respectively. *Periodic training* retrains the model $|S|$ times with an interval of $(k/|S|)$ during evolution, resulting in resource waste and sub-optimal results from retraining. Once a series of graph events have been evaluated, our method can detect instantly the degree of influence they have on the graph. Defining $\Delta \mathbf{Z} = \|\mathbf{Z}_i - \mathbf{Z}_{i-1}\|_F$, $1 \leq i \leq k$, we can track the degree of change for the current feature matrix compared to the previous one. Therefore, we propose an *Adaptive training* strategy based on InstantGNN to control the interval width and obtain better performance in budget-constrained model retraining. Specifically, we consider that we should retrain the model only when a significant accumulative change is present, in other words, when the combined impact of numerous events surpasses a certain threshold, denoted as θ .

For graphs that satisfy specific graph change patterns, we can easily predict when to retrain models next time by fitting their curves. Specifically, for a graph that exhibits changes that follow a certain distribution, after finding the first r times for retraining under the guidance of θ , we can fit the curve to predict remaining $(|S| - r)$ times when the model should be retrained. As a result, we can retrain the model frequently when the graph changes dramatically and often, while increasing the interval between retraining when the graph changes gently. **For generic graphs,** since there is no prior knowledge of the graph change patterns, the problem turns into choosing the appropriate θ . In the real-world, graph evolution is a long-term process. A feasible solution is to compute the average $\Delta \mathbf{Z}$ over the first few small intervals (e.g., first ten intervals of 10,000 graph events) and use it as the threshold θ to determine when the model needs to be retrained in the future.

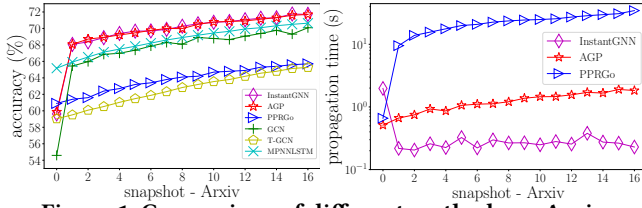


Figure 1: Comparison of different methods on Arxiv.

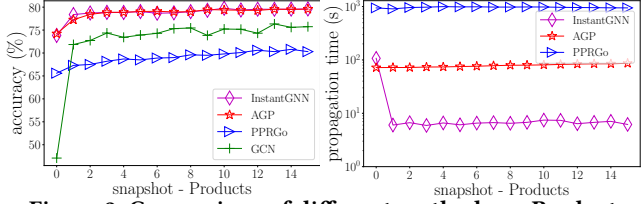


Figure 2: Comparison of different methods on Product.

5 EXPERIMENTS

In this section, we conduct experiments on both real-world and synthetic datasets to verify the effectiveness and efficiency of InstantGNN. First, we compare the accuracy and efficiency of InstantGNN with the state-of-the-art methods on graphs with static labels and dynamic labels, respectively. Then, we conduct an experiment to demonstrate that InstantGNN is capable of adaptive training and achieving superior results while maintaining the same budget with periodic training.

5.1 Datasets and Baselines

Datasets. We use three public datasets in different sizes, provided by [14], for our experiments: a citation network Arxiv, a products co-purchasing network Products and a large-scale citation network Papers100M. To imitate the graphs' dynamic nature, we remove a certain number of edges from the original graphs and replace them later. Due to the scarcity of publicly available datasets for dynamic graph node classification, particularly those with dynamic labels, we construct a real-world dataset and four synthetic datasets with dynamic labels, to better evaluate the effectiveness of our method and facilitate the evolution of dynamic GNNs. Details of data pre-processing are deferred to the appendix.

- **Aminer.** Following [42], we derive a co-authorship network from the academic social networks of Aminer [29]. It consists of 1,252,095 authors as vertices, and the coauthor relationships are considered as edges. The information about authors' communities is derived from *Australian CORE Conference and Journal Ranking* [2] and used as labels in this dataset.
- **SBM datasets.** We generate graphs with community structure using Stochastic Block Model (SBM) [27]. Based on a similar dynamic strategy with [36], we inherit nodes from the graph at the previous time step and change their connectivity to simulate the evolution of communities in the real world. The features of nodes are generated from sparse random vectors.

Table 2 summarizes the properties and scales of the datasets used in the paper. For the number of edges of these datasets, we report the value at the last snapshot. Note that d denotes the dimension of node features, $|C|$ is the number of categories, and $|S|$ represents the number of snapshots. All graphs in those datasets are transformed into unweighted and undirected graphs.

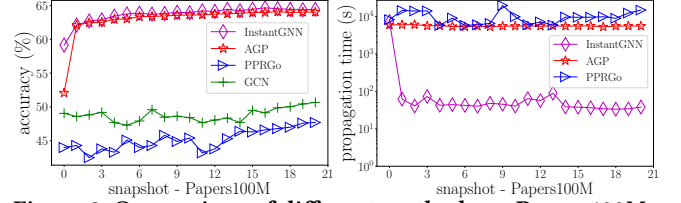


Figure 3: Comparison of different methods on Papers100M.

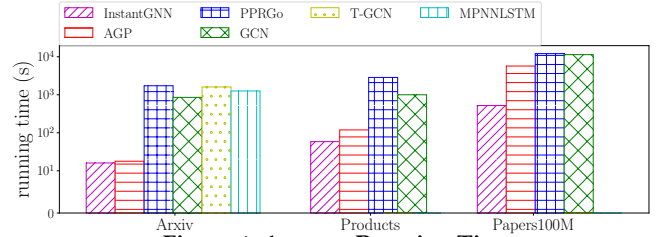


Figure 4: Average Running Time.

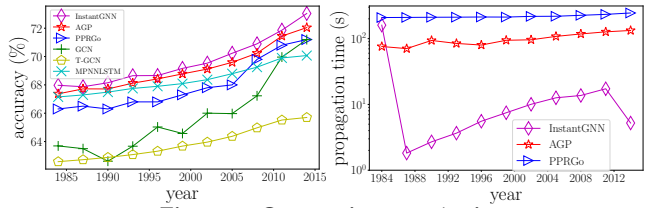


Figure 5: Comparison on Aminer.

We compare the proposed method with several state-of-the-art methods, including static and dynamic graph learning methods. Static methods include GCN [17] and two scalable GNN models, PPRGo [4] and AGP [33], and we train these models at each snapshot. In particular, we adopt GraphSAINT [38] to sample subgraphs so that GCN can be trained on sampled subgraphs and scale to large graphs. We also compare InstantGNN with two temporal graph neural networks, including T-GCN [41] and MPNNLSTM [24]. More details and parameter settings of baseline methods can be found in the technical report. For a fair comparison, we preserve representation matrices of AGP, PPRGo and InstantGNN at the same approximation level, unless specifically stated.

5.2 Evaluation on CTDGs with Static Labels

On Arxiv, Products, and Papers100M, we run experiments using the basic experimental configuration of OGB [14] and the dynamic setting mentioned in Section 5.1. Particularly, nodes in these three datasets have static labels that would not change over time. We compare the accuracy and running time of different methods on each dataset. The time of InstantGNN, AGP and PPRGo can be divided into two parts: the time to obtain the propagation matrix (also known as preprocessing) and the time to train the neural network. Since we focus on reducing the propagation time on dynamic graphs, Figure 1-3 show the comparison results of accuracy and propagation time on Arxiv, Products and Papers100M, respectively. Overall, InstantGNN achieves orders-of-magnitude acceleration without compromising training accuracy, and advantages become even more apparent on large graphs. The results suggest that InstantGNN needs drastically less time to obtain the propagation matrix for graph at the current time, based on the empirical results. For instance, after obtaining the initialized propagation matrix, InstantGNN only spends about 60 seconds updating the propagation

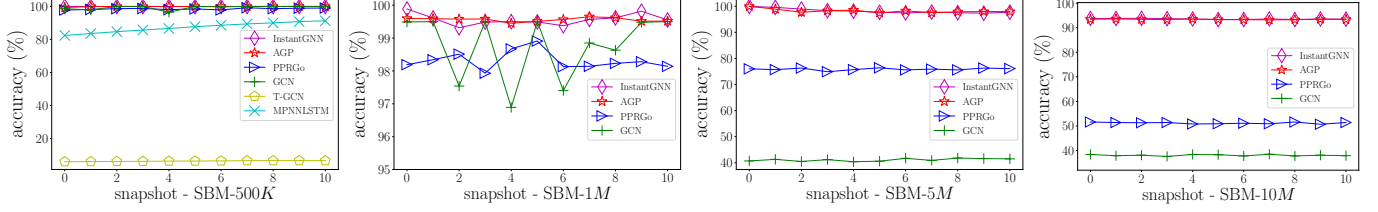


Figure 6: Comparison of accuracy on SBM datasets.

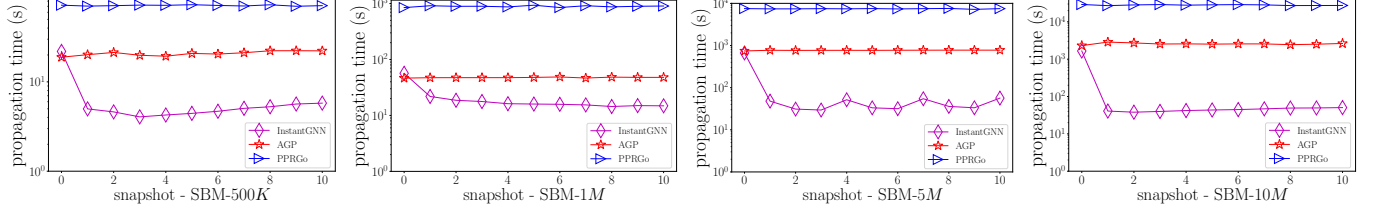


Figure 7: Comparison of propagation time on SBM datasets.

matrix for the next snapshot on Papers100M. In comparison, AGP has to calculate the result from scratch and needs more than an hour. We also observe that it is challenging for T-GCN and MPNNLSTM to scale to large-scale graphs such as Products and Papers100M. To be more specific, they cannot complete one training epoch within 48 hours on Products and run out of 768GB memory on Papers100M. Figure 4 shows the average total running time on three datasets, and we can come to a similar conclusion as above.

5.3 Evaluation on CTDGs with Dynamic Labels

In this section, we conduct experiments on Aminer and four synthetic datasets. The label of each node in these datasets at time i is heavily influenced by its present attributes, which are contained in the current graph G_i . As a result, labels evolve over time, more closely resembling the situation in real-world applications. We split nodes into training, validation, test sets in a 70:10:20 ratio. **Results on Aminer.** Figure 5 shows the comparison in accuracy and propagation time of different methods on Aminer. We observe that InstantGNN can significantly reduce propagation time while achieving better performance. InstantGNN generates updated propagation matrices based on previous computation results, allowing for a $10\times$ speedup over static approaches that require starting from scratch for graph signal propagation and neural network training. We also note that there is an upward trend of propagation time. Understandably, the raw statistical data is collected from 1984, when authors began to collaborate with each other. As the amount of collaboration grows over time, the number of edges grows as well, resulting in an increase in propagation time.

Results on SBM datasets. We also evaluate InstantGNN on four SBM datasets with 500K, 1M, 5M, and 10M nodes ($1K = 10^3$, $1M = 10^6$), respectively. The graph events, which include insertions and deletions of edges, occur at random in these datasets, and the labels of nodes change correspondingly, as discussed in Section 5.1. Figure 6 compares the accuracy of different methods. We observe that InstantGNN, AGP, PPRGo, GCN and MPNNLSTM can capture structural information of graphs even from random features, while T-GCN fails to convergence. Figure 7 plots propagation time on every snapshot. A critical observation is that InstantGNN can achieve comparable accuracy to the strongest baseline AGP while requiring significantly less propagation time. More specifically, on the SBM-10M dataset and snapshot $i > 0$, InstantGNN reduces the

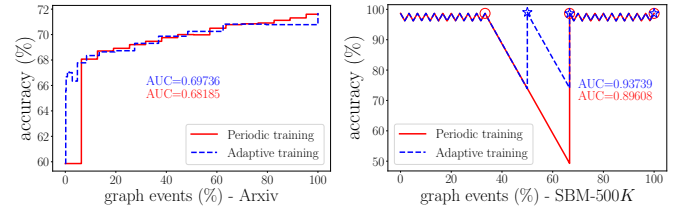


Figure 8: Periodic training vs. Adaptive training.

propagation time by more than 50 times over AGP without sacrificing accuracy. Due to the excessive running time of PPRGo and GCN, on both SBM-5M and SBM-10M datasets, we tune their parameters to optimize the performance and ensure that propagation and training processes could be completed within 48 hours.

5.4 Evaluation of Adaptive Training

We conduct experiments to demonstrate the effectiveness of *Adaptive training* mentioned in Section 4.5, details of the experimental setup can be found in the appendix. Setting the number of retraining times $|S| = 16$, Figure 8 (left) shows the performance of *Adaptive training* compared with *Periodic training* on Arxiv, which has only insertion during the whole graph evolution and has been used in Section 5.2. It is evident that from the results, our method effectively identifies when it should retrain the model to obtain overall better results. As a result, quantitatively, we achieve a 1.5% higher Area Under Curve (AUC) than *Periodic training* over the entire course of the graph evolution. We also evaluate Adaptive training on SBM-500K with both inserted and deleted edges during the evolution, which better simulates real networks. Setting $|S| = 3$, Figure 8 (right) plots the accuracy curves of *Periodic training* and *Adaptive training*, where hollow markers denote the moment of retraining. We observe that *Adaptive training* significantly outperforms *Periodic training* with a 4.1% higher AUC.

6 CONCLUSION

This paper presents an instant graph neural network for dynamic graphs, which enables the incremental computation of the graph feature matrix, based on that from the previous step and a graph event. Compared with existing methods, InstantGNN avoids repeated and redundant computations when the graph is updated and enables instant prediction of node properties at the current moment.

Furthermore, InstantGNN achieves adaptive training to maximize the model's performance gains with a given training budget. It also deals with both CTDGs and DA-CTDGs. Extensive empirical evaluation demonstrates that InstantGNN is efficient for training on dynamic graphs while providing better performance. It truly enables online or instant graph neural networks for large-scale dynamic graphs with frequent updates.

ACKNOWLEDGMENTS

This research was supported in part by Beijing Natural Science Foundation (No. 4222028), by National Natural Science Foundation of China (No. 61972401, No. 61932001, No. 61832017, No. U1936205), by the major key project of PCL (PCL2021A12), by Beijing Outstanding Young Scientist Program No. BJJWZYJH012019100020098, by Alibaba Group through Alibaba Innovative Research Program, by CCF-Baidu Open Fund (No.2021PP15002000), by China Unicom Innovation Ecological Cooperation Plan and by the Huawei-Renmin University joint program on Information Retrieval. Additionally, Jiajun Liu was supported in part by CSIRO's Science Leader project R-91559. Sibao Wang was supported by Hong Kong RGC ECS (No. 24203419), Hong Kong RGC CRF (No. C4158-20G), Hong Kong ITC ITF (No. MRP/071/20X), CUHK Direct Grant (No. 4055181). We also wish to acknowledge the support provided by Intelligent Social Governance Interdisciplinary Platform, Major Innovation & Planning Interdisciplinary Platform for the "Double-First Class" Initiative, Public Policy and Decision-making Research Lab, Public Computing Cloud, Renmin University of China.

REFERENCES

- [1] <http://arxiv.org/abs/2206.01379>.
- [2] <https://www.core.edu.au/>.
- [3] Reid Andersen, Christian Borgs, Jennifer Chayes, John Hopcraft, Vahab S Mirrokni, and Shang-Hua Teng. Local computation of pagerank contributions. In *International Workshop on Algorithms and Models for the Web-Graph*, pages 150–165, 2007.
- [4] Aleksandar Bojchevski, Johannes Klicpera, Bryan Perozzi, Amol Kapoor, Martin Blais, Benedek R  zemberczki, Michal Lukasik, and Stephan G  nnemann. Scaling graph neural networks with approximate pagerank. In *KDD*, pages 2464–2473, 2020.
- [5] Jianfei Chen, Jun Zhu, and Le Song. Stochastic training of graph convolutional networks with variance reduction. In *ICML*, pages 942–950, 2018.
- [6] Jie Chen, Tengfei Ma, and Cao Xiao. Fastgcn: Fast learning with graph convolutional networks via importance sampling. In *ICLR*, 2018.
- [7] Ming Chen, Zhewei Wei, Bolin Ding, Yaliang Li, Ye Yuan, Xiaoyong Du, and Ji-Rong Wen. Scalable graph neural networks via bidirectional propagation. In *NeurIPS*, 2020.
- [8] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks. In *KDD*, pages 257–266, 2019.
- [9] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. Graph neural networks for social recommendation. In *WWW*, pages 417–426, 2019.
- [10] Palash Goyal, Sujit Rokka Chhetri, and Arquimedes Canedo. Dyngraph2vec: Capturing network dynamics using dynamic graph representation learning. *Knowledge-Based Systems*, 187:104816, 2020.
- [11] Palash Goyal, Nitin Kamra, Xinran He, and Yan Liu. Dyngem: Deep embedding method for dynamic graphs. *arXiv preprint arXiv:1805.11273*, 2018.
- [12] Pankaj Gupta, Ashish Goel, Jimmy Lin, Aneesh Sharma, Dong Wang, and Reza Zadeh. Wtf: The who to follow service at twitter. In *Proceedings of the 22nd international conference on World Wide Web*, pages 505–514, 2013.
- [13] William L Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *NeurIPS*, pages 1025–1035, 2017.
- [14] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. *arXiv preprint arXiv:2005.00687*, 2020.
- [15] Glen Jeh and Jennifer Widom. Scaling personalized web search. In *Proceedings of the 12th international conference on World Wide Web*, pages 271–279, 2003.
- [16] Seyed Mehran Kazemi, Rishab Goel, Kshitij Jain, Ivan Kobyzev, Akshay Sethi, Peter Forsyth, and Pascal Poupart. Representation learning for dynamic graphs: A survey. *JMLR*, 21(70):1–73, 2020.
- [17] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.
- [18] Johannes Klicpera, Aleksandar Bojchevski, and Stephan G  nnemann. Predict then propagate: Graph neural networks meet personalized pagerank. In *ICLR*, 2019.
- [19] Johannes Klicpera, Stefan Wei  nberger, and Stephan G  nnemann. Diffusion improves graph learning. In *NeurIPS*, pages 13354–13366, 2019.
- [20] Srijan Kumar, Xikun Zhang, and Jure Leskovec. Predicting dynamic embedding trajectory in temporal interaction networks. In *KDD*, pages 1269–1278, 2019.
- [21] Franco Manessi, Alessandro Rozza, and Mario Manzo. Dynamic graph convolutional networks. *Pattern Recognition*, 97:107000, 2020.
- [22] Giang Hoang Nguyen, John Boaz Lee, Ryan A Rossi, Nesreen K Ahmed, Eunye Koh, and Sungchul Kim. Continuous-time dynamic network embeddings. In *WWW*, pages 969–976, 2018.
- [23] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. Technical report, Stanford InfoLab, 1999.
- [24] George Panagopoulos, Giannis Nikolentzos, and Michalis Vazirgiannis. Transfer graph neural networks for pandemic forecasting. In *AAAI*, volume 35, pages 4838–4845, 2021.
- [25] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao Schardl, and Charles Leiserson. Evolvegcn: Evolving graph convolutional networks for dynamic graphs. In *AAAI*, volume 34, pages 5363–5370, 2020.
- [26] Aravind Sankar, Yanhong Wu, Liang Gou, Wei Zhang, and Hao Yang. Dysat: Deep neural representation learning on dynamic graphs via self-attention networks. In *WSDM*, pages 519–527, 2020.
- [27] Tom AB Snijders and Krzysztof Nowicki. Estimation and prediction for stochastic blockmodels for graphs with latent block structure. *Journal of classification*, 14(1):75–100, 1997.
- [28] Aynaz Taheri, Kevin Gimpel, and Tanya Berger-Wolf. Learning to represent the evolution of dynamic graphs with recurrent models. In *WWW*, pages 301–307, 2019.
- [29] Jie Tang, Jing Zhang, Limin Yao, Juanzi Li, Li Zhang, and Zhong Su. Arnetminer: extraction and mining of academic social networks. In *KDD*, pages 990–998, 2008.
- [30] Rakshit Trivedi, Mehrdad Farajtabar, Prasenjeet Biswal, and Hongyuan Zha. Dyrep: Learning representations over dynamic graphs. In *ICLR*, 2019.
- [31] Petar Veli  kovi  , Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. In *ICLR*, 2018.
- [32] Binghui Wang and Neil Zhenqiang Gong. Attacking graph-based classification via manipulating the graph structure. In *SIGSAC*, pages 2023–2040, 2019.
- [33] Hanzhi Wang, Mingguo He, Zhewei Wei, Sibao Wang, Ye Yuan, Xiaoyong Du, and Ji-Rong Wen. Approximate graph propagation. In *KDD*, page 1686–1696, 2021.
- [34] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. Simplifying graph convolutional networks. In *ICML*, pages 6861–6871, 2019.
- [35] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. In *ICML*, pages 5453–5462, 2018.
- [36] Tianbao Yang, Yun Chi, Shenghuo Zhu, Yihong Gong, and Rong Jin. Detecting communities and their evolutions in dynamic social networks—a bayesian approach. *Machine learning*, 82(2):157–189, 2011.
- [37] Jiaxuan You, Bowen Liu, Rex Ying, Vijay Pande, and Jure Leskovec. Graph convolutional policy network for goal-directed molecular graph generation. In *NeurIPS*, pages 6412–6422, 2018.
- [38] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. Graphsaint: Graph sampling based inductive learning method. In *ICLR*, 2019.
- [39] Chen Zhang, Qiuchi Li, and Dawei Song. Aspect-based sentiment classification with aspect-specific graph convolutional networks. In *EMNLP-IJCNLP*, pages 4568–4578, 2019.
- [40] Hongyang Zhang, Peter Lofgren, and Ashish Goel. Approximate personalized pagerank on dynamic graphs. In *KDD*, pages 1315–1324, 2016.
- [41] Ling Zhao, Yujiao Song, Chao Zhang, Yu Liu, Pu Wang, Tao Lin, Min Deng, and Haifeng Li. T-gcn: A temporal graph convolutional network for traffic prediction. *IEEE Transactions on Intelligent Transportation Systems*, 21(9):3848–3858, 2019.
- [42] Lekui Zhou, Yang Yang, Xiang Ren, Fei Wu, and Yueting Zhuang. Dynamic network embedding by modeling triadic closure process. In *AAAI*, 2018.
- [43] Meiqi Zhu, Xiao Wang, Chuan Shi, Houye Ji, and Peng Cui. Interpreting and unifying graph neural networks with an optimization framework. In *WWW*, pages 1215–1226, 2021.
- [44] Yuan Zuo, Guannan Liu, Hao Lin, Jia Guo, Xiaoqian Hu, and Junjie Wu. Embedding temporal network via neighborhood formation. In *KDD*, pages 2857–2866, 2018.

A BATCH UPDATE

We show how to construct estimated vectors for dynamic graphs with one-by-one updated edges in Algorithm 2. We adapt our approach in this section to support parallel processing and batch update settings. We begin with an equivalent approach of updating a single node. That is, we can modify the estimate of a node and so avoid repeatedly updating residuals on its neighbors. Taking the insertion of edge $e = (u, v)$ as an example, as described in Section 4.2, changes in the degrees of nodes u and v have an effect on their neighbors to maintain their equations hold. The primary cause is the transition of $\frac{\hat{\pi}(u)}{d(u)^{1-\beta}}$ to $\frac{\hat{\pi}(u)}{(d(u)+1)^{1-\beta}}$. As stated in line 5-6 and 9-19 of Algorithm 2, we add the increment $\hat{\pi}(u) \cdot (\frac{1}{(d(u)+1)^{1-\beta}} - \frac{1}{d(u)^{1-\beta}})$ and $\hat{\pi}(v) \cdot (\frac{1}{(d(v)+1)^{1-\beta}} - \frac{1}{d(v)^{1-\beta}})$ to the residuals of each node $w \in N(u)$ and $y \in N(v)$. To avoid modifying all neighbors of nodes u and v , we let $\hat{\pi}'(u) = \hat{\pi}(u) \cdot \frac{(d(u)+1)^{1-\beta}}{d(u)^{1-\beta}}$, where $\hat{\pi}'(u)$ denotes the updated estimate of node u . As a result, we have $\frac{\hat{\pi}'(u)}{(d(u)+1)^{1-\beta}} = \frac{\hat{\pi}(u)}{d(u)^{1-\beta}}$, and the equation for node $w \in N(u)$ remains valid. Additionally, the residual of node u needs to be adjusted to account for the change in $\hat{\pi}(u)$, i.e., $\Delta r(u) = -\frac{\hat{\pi}'(u) - \hat{\pi}(u)}{\alpha} = \hat{\pi}'(u) \cdot \frac{d(u)^{1-\beta} - (d(u)+1)^{1-\beta}}{\alpha \cdot (d(u)+1)^{1-\beta}}$. Let $P(w, u)$ be the element of the w -th row and u -th column of transition matrix P . Note that $\Delta r(u) \cdot (1 - \alpha)P(w, u) = \Delta r(w)$ (described in Section 4.2), indicating that $\Delta r(u)$ can be transformed into $\Delta r(w)$ following another push operation. This demonstrates the equivalent nature of these two operations from another perspective. We omit details about updating node v since it is handled similarly. The advantage of modifying $\hat{\pi}$ is that we need only update the nodes u and v when edge $e = (u, v)$ arrives, simplifying subsequent parallel processing.

Here we focus on the dynamic propagation algorithm under batch update setting, and Algorithm 3 gives the pseudo-code. The main idea is to simultaneously calculate the increments caused by multiple edges. Given a set of graph events $M = \{Event_1, \dots, Event_k\}$, line 3 corresponds to the graph updating stage, where the graph in time step k , G_k , is generated from the initial graph guided by M , and k denotes the batch size. Lines 5-9 represent the first step of updating the affected nodes, in which we update the estimates and residuals of these nodes in the way aforementioned. Recall that a portion of an affected node's increment is caused by its added or deleted neighbors. Therefore, we compute the increment by iterating over all new and vanishing neighbors of each affected node, as in lines 10-17. Line 18 invokes the basic propagation algorithm to reduce the error, which is the same as line 7 in Algorithm 2.

B ERROR ANALYSIS

To analyze the error bound, we first note that Algorithm 1 is applied to obtain the estimate vector $\hat{\pi}_i$ and residual vector r_i corresponding to G_i after computing increments, as line 7 in Algorithm 2. This means that the error adjustment of $\hat{\pi}_i$ is actually performed by Algorithm 1. Therefore, the dynamic propagation method can preserve the result with the same error for each time step. Given the error threshold ε , we use the following theorem to derive the error bound of our method.

THEOREM 5 (ERROR ANALYSIS). *Let $\pi_i(s)$ be the propagation vector of the graph G_i , and $d_i(s)$ is the degree of node s at time i . For*

Algorithm 3: Parallel Dynamic Propagation Algorithm for Batch Update.

Input : Dynamic graph $\{G_0, M = \{Event_1, \dots, Event_k\}\}$, graph signal vector x , error threshold ε , teleport probability α , convolutional coefficient β

Output : estimated propagation vector $\hat{\pi}$, residual vector r

```

1  $\hat{\pi} \leftarrow 0, r \leftarrow x;$ 
2  $\hat{\pi}, r \leftarrow BasicPropagation(G_0, \varepsilon, \alpha, \beta, \hat{\pi}, r);$ 
3 Generate  $G_k$  from updating  $G_0$  by  $M$ ;
4  $V_{affected} \leftarrow \{u \mid \text{the degree of node } u \text{ has changed}\};$ 
5 parallel for each  $u \in V_{affected}$  do
6    $\Delta d(u) \leftarrow \text{the degree to which node } u \text{ has changed};$ 
7    $\hat{\pi}(u) \leftarrow \hat{\pi}(u) \cdot \frac{d(u)^{1-\beta}}{(d(u) - \Delta d(u))^{1-\beta}};$ 
8    $\Delta r_1(u) \leftarrow \hat{\pi}(u) \cdot \frac{(d(u) - \Delta d(u))^{1-\beta} - d(u)^{1-\beta}}{\alpha \cdot d(u)^{1-\beta}};$ 
9    $r(u) \leftarrow r(u) + \Delta r_1(u);$ 
10 parallel for each  $u \in V_{affected}$  do
11    $\Delta r_2(u) \leftarrow (\hat{\pi}(u) + \alpha r(u) - \alpha x(u)) \cdot \frac{(d(u) - \Delta d(u))^\beta - d(u)^\beta}{d(u)^\beta};$ 
12   for each  $v \in \{\text{the added neighbors of node } u\}$  do
13      $\Delta r_2(u) \leftarrow \Delta r_2(u) + \frac{(1-\alpha)\hat{\pi}(v)}{d(u)^\beta d(v)^{1-\beta}};$ 
14   for each  $v \in \{\text{the deleted neighbors of node } u\}$  do
15      $\Delta r_2(u) \leftarrow \Delta r_2(u) - \frac{(1-\alpha)\hat{\pi}(v)}{d(u)^\beta d(v)^{1-\beta}};$ 
16    $\Delta r_2(u) \leftarrow \Delta r_2(u) / \alpha;$ 
17    $r(u) \leftarrow r(u) + \Delta r_2(u);$ 
18  $\hat{\pi}, r \leftarrow BasicPropagation(G_k, \varepsilon, \alpha, \beta, \hat{\pi}, r);$ 
19 return  $\hat{\pi}, r;$ 
```

each node $s \in V$, Algorithm 2 computes estimators $\hat{\pi}_i(s)$ such that $|\hat{\pi}_i(s) - \pi_i(s)| \leq \varepsilon \cdot d_i(s)^{1-\beta}$ holds for $\forall i \in \{0, 1, 2, \dots, k\}$.

C EXPERIMENTAL DETAILS

C.1 Details of data pre-processing

OGB datasets. According to [32], the absence of edges has a detrimental effect on the performance of GNNs, such as decreasing classification accuracy. In contrast, we suppose that accuracy would grow proportionately when these missing edges are gradually reinstated. We use three OGB datasets in different sizes: (1) **Papers100M** is the largest publicly available dataset by far. To imitate the graph's dynamic nature, we eliminated 40 million edges from the original dataset and reinserted them line by line later. (2) **Arxiv** is also a citation network. It is constructed similarly to Papers100M, except that we remove about 950 thousand edges at the start. (3) **Products** is a graph generated from the Amazon product co-purchasing network. We construct the dynamic graph in the same way except that the number of deleting edges is set to 30 million. We utilized the node features and labels provided by OGB [14].

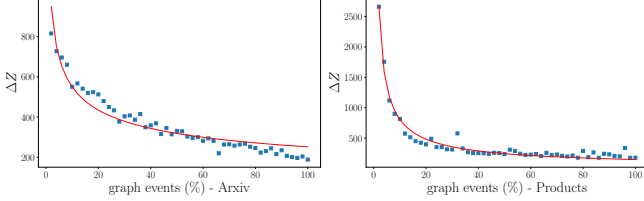
Aminer. In this dataset, time steps are defined as consecutive non-overlapping one-year periods. At each time step i , if an author had published papers collaborating with another, we create an edge between them in G_i , whose weight is ignored. With this definition,

Table 3: Parameters of generating SBM datasets.

Dataset	#nodes	#classes	#intra-block edges	#inter-block edges	#snapshots	#nodes changed per snapshot
SBM-500K	500,000	50	20	1	10	2500
SBM-1M	1,000,000	50	20	1	10	2500
SBM-5M	5,000,000	100	20	1	10	2500
SBM-10M	10,000,000	100	20	1	10	2500

Table 4: Parameters of InstantGNN.

Dataset	β	α	ϵ	learning rate	batch size	dropout	hidden size
Arxiv	0.5	0.1	$1e-7$	0.0001	10000	0.3	1024
Products	0.5	0.1	$1e-7$	0.0001	10000	0.2	1024
Papers100M	0.5	0.2	$1e-9$	0.0001	10000	0.3	2048
Aminer	0.5	0.1	$1e-7$	0.0001	1024	0.1	256
SBM-500K	0.5	0.001	$1e-7$	0.01	1024	0.1	1024
SBM-1M	0.5	0.001	$1e-8$	0.001	4096	0.1	1024
SBM-5M	0.5	0.001	$1e-8$	0.001	8192	0.3	2048
SBM-10M	0.5	0.001	$5e-9$	0.001	8192	0.3	2048

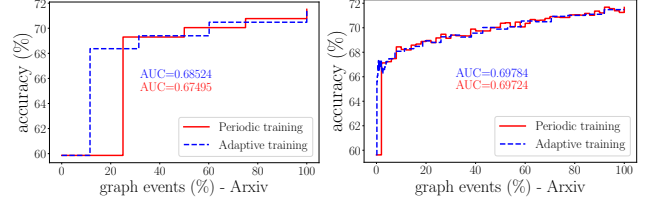
**Figure 9: Fit ΔZ follows a power-law distribution on various datasets.**

we obtain 1,267,755 – 4,025,865 edges, evolving from 1984 to 2014. Furthermore, we say an author belongs to a particular community if over half of his or her papers were published in corresponding journals this year. Recall that the community the author belongs to is used as the label. Therefore, if an author publishes in a different field each year, the label will change accordingly with the time step. In addition, we generate node features by utilizing the BERT-as-service to extract word vectors from the authors’ descriptions.

SBM datasets. We propose a study on synthetic graphs using a dynamical SBM. The SBM model will decide the relationship between nodes stochastically using probabilities based on the given community memberships. Given a graph G_{i-1} with the label set C_{i-1} at time step $i-1$, we generate the graph G_i by migrating some of the nodes. Specifically, we randomly select k nodes to form a subset of V , denoted as $\bar{V}$. For each node $s \in \bar{V}$, we assume it belongs to the community $y \in C_{i-1}$ at time step $i-1$. Then it turns to be a member of community $y' \in \{C_i - y\}$ by removing the edges connected to the community y and adding the connection to nodes of community y' . Table 3 provides detailed parameters for generating the four SBM datasets we used in Section 5.3.

C.2 More details of adaptive training

Firstly, we perform an empirical evaluation on Arxiv. Setting $|S| = 16$, we can retrain the model 16 times, while the model is forced to retrain after k -th graph event, where k is the total number of graph events. For *Periodic training*, we retrain the model in a fixed interval, e.g. every $k/|S|$ graph events, and the accuracy curve is shown as the red line in Figure 8 (left). For *Adaptive training*, we found that ΔZ of Arxiv follows a power-law curve during the evolution. An intuitive explanation is that the graph becomes dense after a series

**Figure 10: Adaptive training on Arxiv setting $|S|=4$ and $|S|=50$.**

of edges have been inserted. The changes triggered by continued insertion become smaller and smaller. Products, another insertion only dataset used in Section 5.2 also shows a similar property, as shown in Figure 9. We set $\theta = \frac{\Delta Z}{\|Z_t - Z_0\|_F}$ to determine the first $r = 3$ times to retrain the model, and the remaining $(|S| - r = 13)$ retraining time is determined by fitting the power-law curve. *Adaptive training* achieves a better retraining gain based on the qualitative and quantitative results, as shown in Figure 8 (left). Figure 10 shows the comparison results of setting $|S| = 4$ and $|S| = 50$, and we can reach similar conclusions. Setting $r = 3$, we set $\theta = 0.05$ for $|S| = 16, 50$ and $\theta = 0.3$ for $|S| = 4$.

We also evaluate *Adaptive training* on SBM-500K with both inserted and deleted edges during the evolution, which better simulates real networks. We consider a special case where several nodes change their communities and recover immediately, and only a fraction of the changes are permanent. Figure 8 (right) shows the accuracy curves of setting $|S| = 3$ and $\theta = \Delta Z = 1.73$. Note that we set θ empirically here, to maintain the fixed budget for a fair comparison, we make the model stop training after reaching the budget constraint.

C.3 Implementation details

We implement InstantGNN in PyTorch and C++. Each layer is activated using the ReLU non-linear function. Additionally, a dropout function is used behind each layer. Table 4 summarizes parameters settings of InstantGNN. For baseline methods, we employ their published codes or the codes implemented with Pytorch Geometric Temporal. All experiments are completed on a machine with an NVIDIA RTX A6000 GPU (48GB memory), Intel Xeon CPU (2.20 GHz) with 40 cores, and 768GB of RAM.